Assignment 1: Digital Payment System

Core Idea

Design a system where different payment methods follow different processing rules but share a common structure.

Problem Statement

You are building a Digital Payment System.

There are multiple payment types:

- CreditCardPayment
- UPIPayment
- WalletPayment

All payments must:

- Validate payment amount (> 0)
- Process payment
- Generate receipt

However:

- Each payment type processes transaction differently.
- Credit card charges 2% processing fee.
- UPI has no fee.
- Wallet has 1% processing fee.

Requirements

1. Create an abstract class `Payment`.
2. It must contain:
    - protected amount
    - constructor to initialize amount
    - concrete method validateAmount()
    - abstract method processPayment()
    - concrete method generateReceipt()
3. Child classes must:

    - Implement processPayment() differently
    - Optionally override generateReceipt() if needed
4. Do not use if-else to decide payment type.
5. In main:
    - Create array of Payment objects
    - Process each payment polymorphically

Assignment 2: Hospital Billing System

Core Idea

Understand template pattern using abstract class.

Problem Statement

Design a Hospital Billing System.

Patients can be:

- InPatient
- OutPatient
- EmergencyPatient

Every patient:

- Has patientId and name.
- Needs bill generation.
- Needs tax calculation.
- Needs final amount computation.

However:

- InPatient includes room charges.
- OutPatient includes consultation fee.
- EmergencyPatient has emergency surcharge.

Requirements

1. Create an abstract class `Patient`.
2. Include:
    - private patientId
    - private name
    - constructor with encapsulation
    - concrete method generateBill()
    - abstract method calculateCharges()
3. generateBill() should:
    - Call calculateCharges()

    - Add tax (common to all)
    - Print final amount
4. Child classes must:
    - Implement calculateCharges()
5. Store mixed patient types in Patient[] array and process bills.

Assignment 3: Transport Fare Calculation Engine

Core Idea

Model a system where calculation logic varies but structure remains same.

Problem Statement

Build a Fare Calculation Engine for public transport.

Transport types:

- Bus
- Metro
- Taxi

Each transport:

- Has routeId
- Has baseFare

Every transport must:

- CalculateFare()
- PrintTicket()

However:

- Bus fare depends on distance.
- Metro fare depends on number of stations.
- Taxi fare depends on distance + time.

Requirements

1. Create abstract class `Transport`.
2. Include:
    - protected routeId
    - protected baseFare

    - constructor to initialize
    - concrete method printTicket()
    - abstract method calculateFare()
3. Child classes must:
    - Implement calculateFare()
4. Use Transport[] array and loop to calculate fare.
5. Demonstrate constructor chaining.
6. Avoid type checking using if-else.